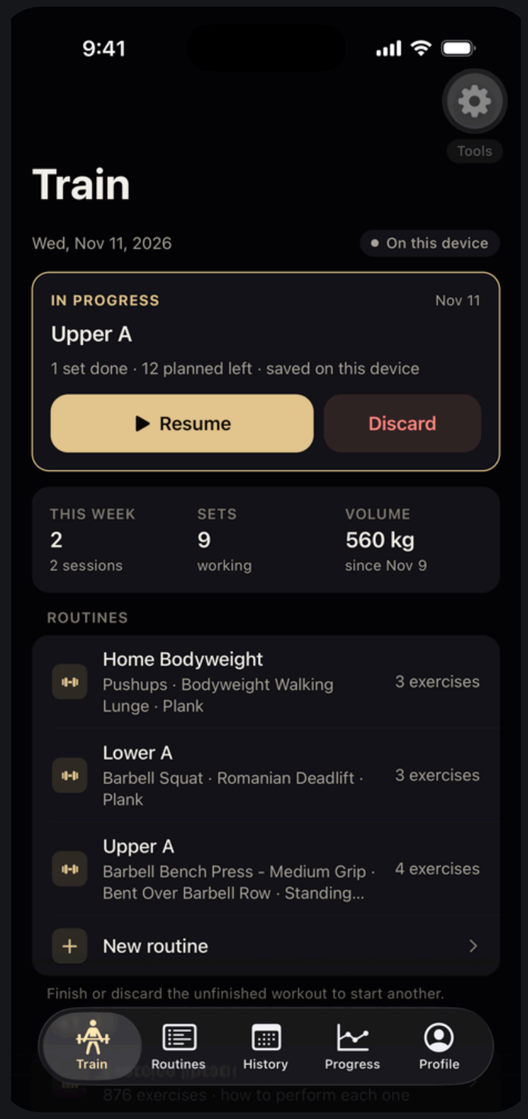
CS3365 SOFTWARE ENGINEERING · PROJECT 2

From diagrams to a one-shot app

Fitness & Workout Log: the design artifacts, how we iterated them, and a live demo of two generated apps, one built by OpenAI and one by Claude.

Sebastian · Diego · November 12, 2026



OUR SELECTION · OPTION 1 OF 4

We chose the Claude Fitness & Workout Log

People who train regularly log sets in notes, paper or spreadsheets. Names drift, last week’s numbers are hard to find at the rack, and nobody can see whether they are actually improving.

31
user stories, FW-01 to FW-31

101
story points, 9 one-pagers

After the professor’s review, personas are named by role only, and every story names a role.

Experienced lifter
Trains five days a week. Needs fast entry between sets, consistent names and volume graphs.

Beginner lifter
Three months in. Needs last time’s weights, a simple flow and a plain summary.

Home trainer
Shift worker at home. Needs reps-only and timed sets, and logging with no signal.

Returning lifter
Low tech comfort. Needs large text, kg and lb at two gyms, a readable history.

Source: Fitness_Workout_Log_App_Revised_v3.docx (professor-reviewed 1-pagers) · 2

WHAT WE TOOK FROM THE 1-PAGERS

Six needs that shaped the design

Users need…	So the app…
Some exercises have no weight	only asks for what applies: weight, reps or seconds
Never lose a set, even offline	saves on the phone first and uploads later, never twice
Exercise names stay consistent	gives each exercise a fixed ID, so renaming keeps its history
Fix mistakes after a workout	updates history and graphs everywhere after an edit or delete
Some gyms use kg, others lb	saves the weight as typed and converts only on screen
Text that is easy to read	lets text grow to 200%, and every chart also lists its numbers

We also added planned training: routines like 4 sets of 12, a rest timer on the Lock Screen, warm-ups, 876 exercises, a muscle map, reports and data export.

Source: the nine 1-pagers in Fitness_Workout_Log_App_Revised_v3.docx · 3

THE DESIGN

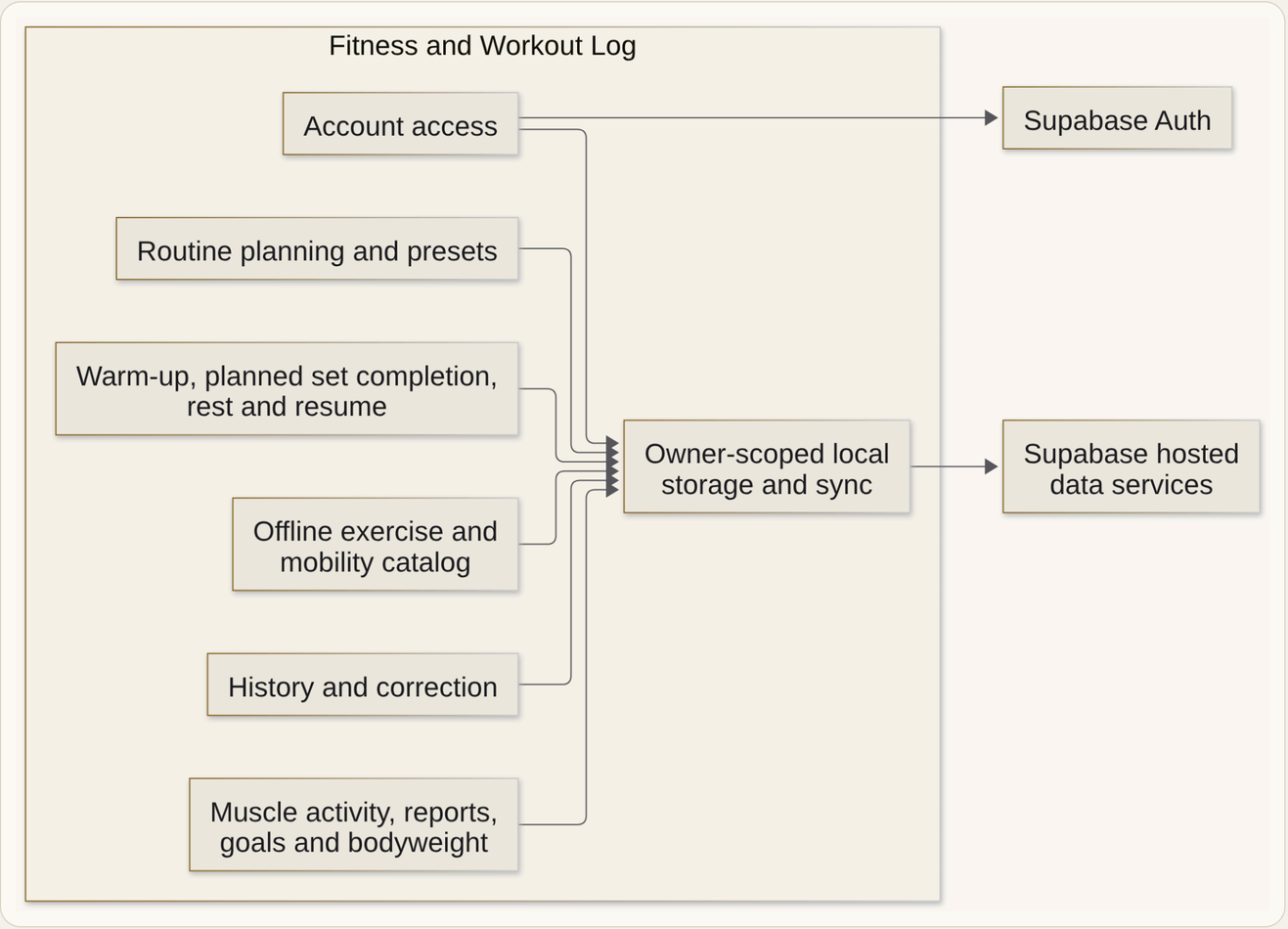
A blueprint of 50 diagrams, each reviewed on GitHub

1	1	40	8
Context model	**Architecture**	**Sequences**	**Structure**
What the app includes and what it connects to	How the phone, the cloud and the app store fit together	Each user story, step by step	Screens, data and classes

- Every one of the 31 user stories has its own step-by-step diagram
- Each diagram went in through its own pull request (#2–#45)
- Next: the four diagrams that matter most

CONTEXT MODEL

Every module shares one data path



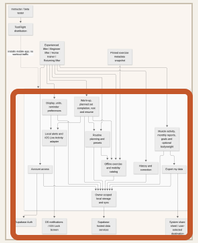
One shared data path. Every part of the app (plans, logging, exercises, history, profile, account) saves through one module.

Private by design. Each record belongs to one signed-in user.

Only two connections out: cloud storage and sign-in, both on Supabase.

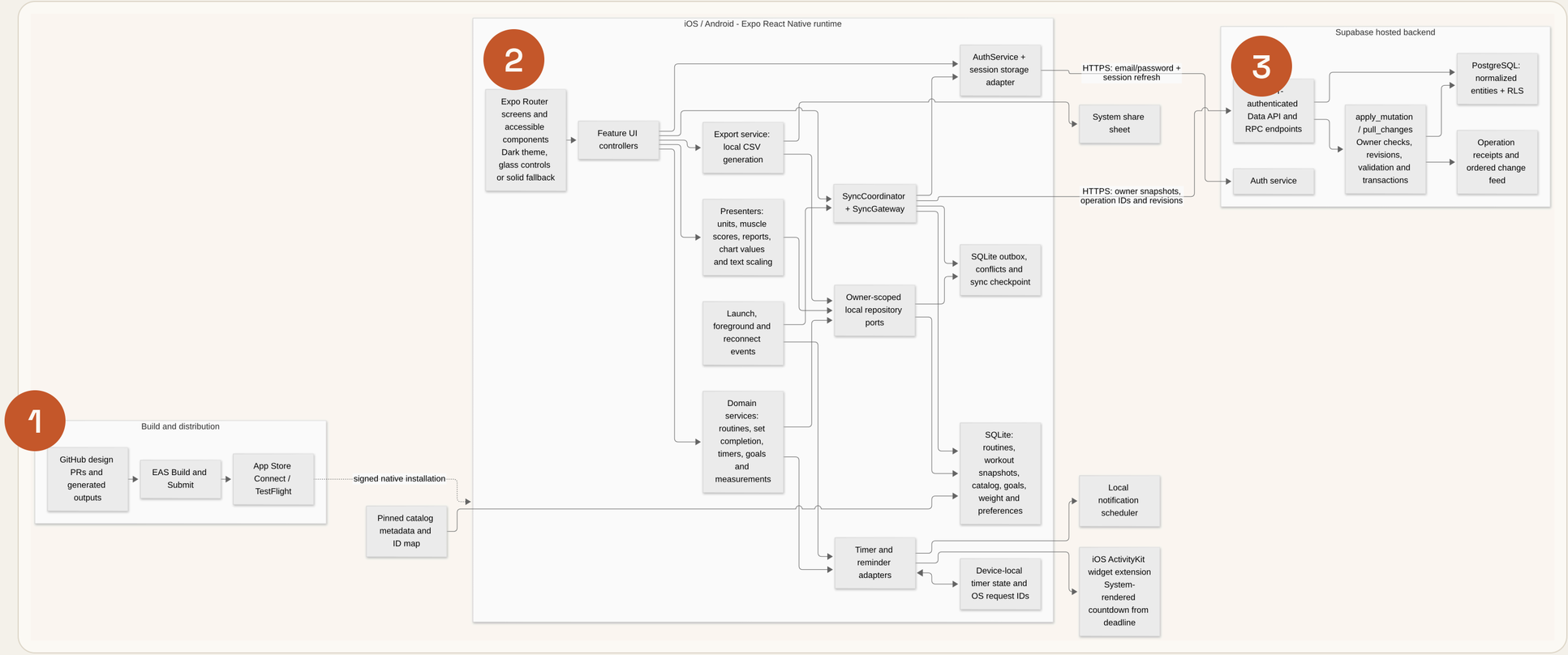
Why it matters: working offline and syncing safely are solved once, in one place.

In the full diagram



ARCHITECTURE · OVERVIEW

Architecture: build, phone and cloud



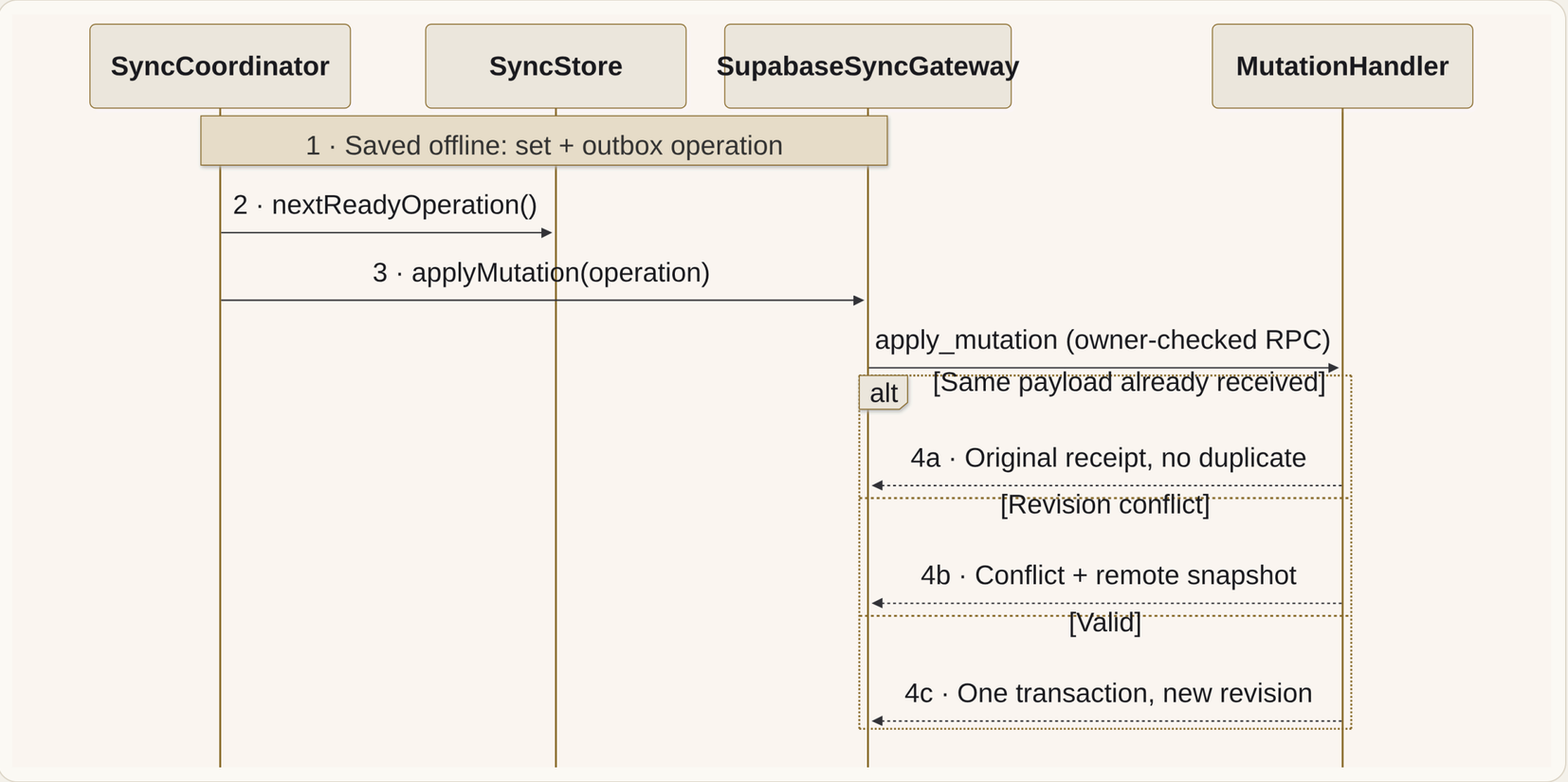
1 Build: GitHub → app build → TestFlight

2 Phone: screens, logic, on-phone storage

3 Cloud: sign-in, sync, database

SEQUENCE DIAGRAM · STORY FW-09 · THE RISKIEST STORY

Logging offline, uploading later



1 · Offline: the set and its upload wait on the phone.

2–3 · Online: the app sends them.

4a · Sent twice? The server spots it: no duplicates.

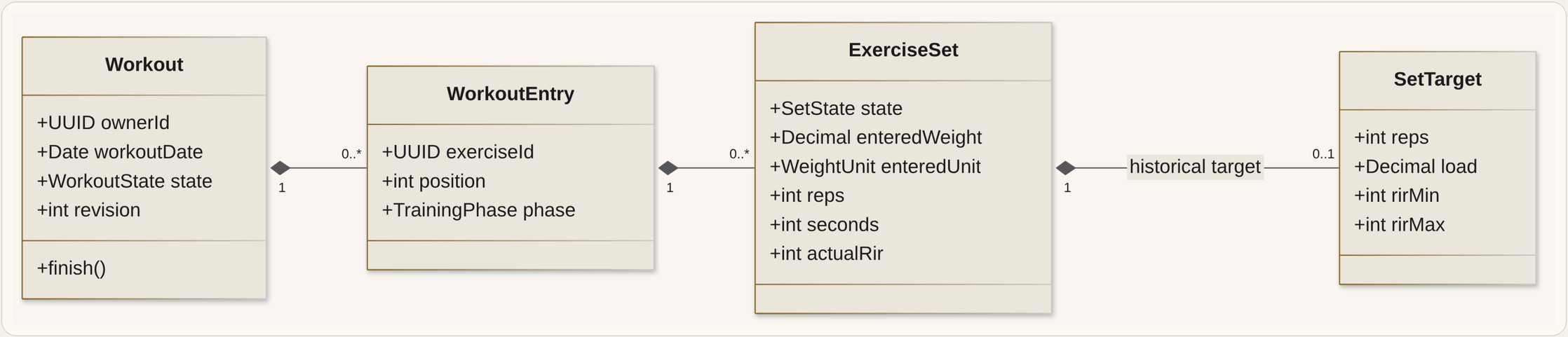
4b · 4c: a clash keeps both versions; otherwise it saves.

In the full diagram



CLASS DIAGRAM · CORE DATA

How a workout is stored



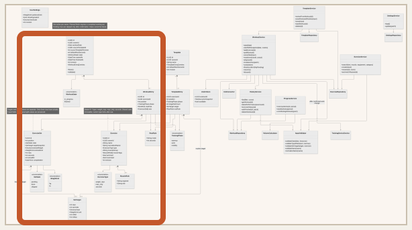
Nested: a workout holds exercises, and each exercise holds its sets.

As typed: each set keeps the weight and unit entered; kg/lb is converted only on screen.

Plan vs. actual: the plan (4 × 12) is stored apart from what was lifted.

Private and synced: every workout records its owner and version.

In the full diagram



THE ITERATION LOOP

We changed diagrams, never code

1	2	3	4	5	6
Revised 1-pagers	Diagrams	Pull request	AI builds the app	We test it	Feedback
after professor feedback	the blueprint	reviewed, then merged	in one session	simulator and TestFlight	becomes new requirements

Step 6 loops back to step 2, up to three rounds per model. Every round is recorded: the model, the design version, the tests, and what is still unchecked.

Faster set entry (BL-06) was fixed separately for the phone release, outside the graded one-shot.

PROMPTS USED TO TRIGGER GENERATION

One long brief, then short round prompts

Round 1 one-shot brief

- Build the whole app in one session, without asking us anything
- Read every document and diagram; track every user story
- Use Expo, on-phone storage and Supabase; no fake screens
- Test it on the iPhone simulator and report honestly
- Include setup, requirements, assumptions and demo guides

Later rounds

OpenAI round 2: “add liquid glass buttons … a more dark, premium aesthetic”

OpenAI round 3: a fresh app from the extended design, with planned training

Claude round 2: “too zoomed in … an actual body … male or female … feels laggy”

MODEL A · OPENAI ASTRA 6.1 · HIGH EFFORT · IN CODEX

Three rounds, ending in Formwork

Round 1 · Nov 4 · PR #46

Input: 31 stories, 42 diagrams

Built: workout logging, history, graphs, templates, kg/lb and offline sync

Verified: 7 automated tests; saved a workout during a server outage and uploaded it after

Round 2 · Nov 8 · PR #49

Input: a darker, premium look with glass-style buttons

Verified: iPhone build and tests passed; strong colour contrast

Skipped: the server outage test

Round 3 · Nov 9 · PR #50

Input: the full blueprint, including planned training

Built: Formwork, a fresh app: workout plans, a lock-screen rest timer, 876 exercises

Feedback: a separate form for every set was too slow at the gym

MODEL B · CLAUDE OPUS 5.5 · HIGH EFFORT · IN CLAUDE CODE

Two rounds, building Repwise

Round 1 · Nov 9 · PR #47

Input: the same blueprint, with the set-entry feedback already noted

Built: quick in-row set entry, rest timer, muscle map, routines, reports, export

Verified: 50 automated tests; cloud backend; TestFlight

Round 2 · Nov 10 · PR #48

Feedback first: denser layout, faster screens, a realistic body figure, exercise guides; 5 diagrams updated

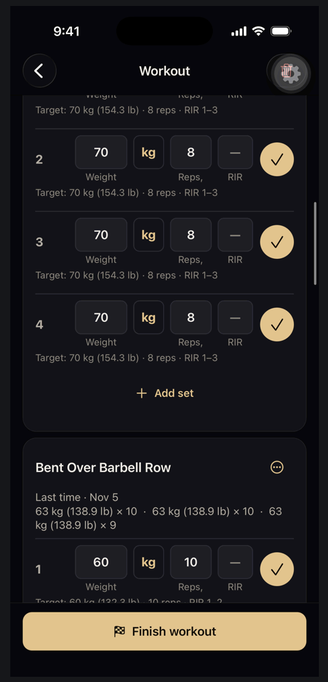
Lag fixed: every set used to reload every screen; now only what changed reloads

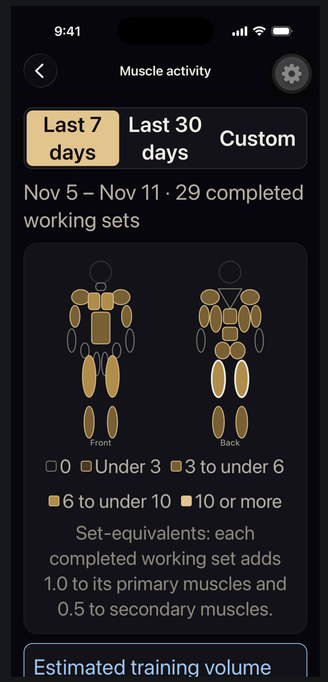
Verified: 53 automated tests passed

Both rounds are saved in the repo as git tags · 12

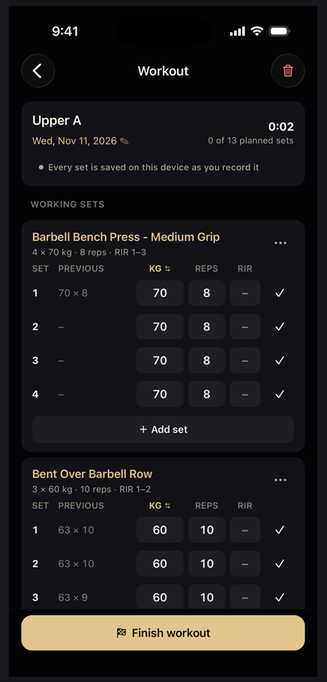
What a diagram change did to the app

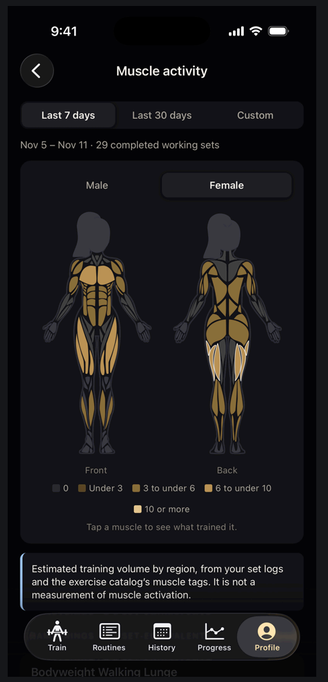
CLAUDE ROUND 1

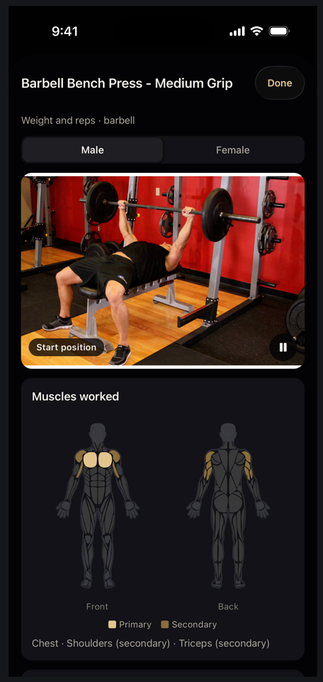




CLAUDE ROUND 2 · UI-04, TR-07A, TR-06A







We changed only the requirements and diagrams. Claude regenerated the code: compact set table, a real body figure, and a How-to-perform guide.

iPhone 17 Pro simulator, iOS 26.5, demo data · 13

Our verdict: Claude’s app was clearly better

	OpenAI · Formwork (round 3)	Claude · Repwise (round 2)
Model	OpenAI Astra 6.1, high effort	Claude Opus 5.5, high effort
Rounds used	3 of 3	2 of 3
Requirements fully verified (model’s own report)	16 of 47	39 of 52
Automated tests	9, plus a database test	53
Offline sync	Worked in an outage test	Worked in cloud tests; conflicts shown
Set entry	A separate form for each set	Right in the list, one tap
Deployment	Cloud backend · TestFlight	Cloud backend · TestFlight

After using both apps, we found Claude’s Repwise much better: nicer to use, more polished, and better overall than OpenAI’s Formwork.

LIVE DEMO · ABOUT 4 MINUTES

One script, two apps

OpenAI · Formwork

1. Open the demo and start the Upper body routine
2. Record 11 reps: the plan still says 12; rest timer starts
3. Close the app and reopen: the workout is still there
4. Finish, then see History and Progress
5. Profile: muscle map and monthly report

Claude · Repwise

1. Start the Upper A routine
2. Record a set right in the row; rest timer starts
3. Close the app and reopen: nothing is lost
4. Finish, then see History and Weekly volume
5. How to perform, and the male/female body map

Backup if Wi-Fi or TestFlight fails: iPhone simulator with demo data, plus the screenshots in the repo.

PRODUCT BACKLOG FOR PHASE 3

Sprint 1 goal: an offline workout you can trust

ID	Story (user · what it fixes)	Points	Sprint 1
BL-01	Home trainer: sets survive closing the app on a real phone	5	**Selected**
BL-02	Home trainer: an offline workout uploads exactly once, conflicts kept	8	**Selected, split**
BL-04	Experienced lifter: corrections show in history and graphs	5	If capacity
BL-03	Returning lifter: kg and lb always shown consistently	5	Later
BL-05	Returning lifter: large text never hides a button	3	Later
BL-06	Lifter: enter weight and reps right in the row	[__]	Device check

Following Sommerville: each item is a user story with acceptance checks, sized in story points and ranked by value and risk. Sprint 1 comes first because every graph depends on a record you can trust.

WHAT WE LEARNED

Diagrams set the structure; using the app set the feel

Diagrams as a contract

Both AI models built the same structure from them

Feel needs a phone

Layout, speed and set entry only showed up in real use, and became new requirements

Evidence over claims

Every round records its model, version, tests and open items

Questions?

github.com/SebasLopez2005/SWEProject1